OTTO: COMPLETE END-TO-END PRODUCT OVERVIEW

From Voice Notes, Clarifications, and Technical Discussions

EXECUTIVE SUMMARY

Otto is an AI-orchestrated sales execution platform for home services companies that converts unstructured customer interactions (calls, texts, field visits) into structured, stage-based workflows with hybrid AI-human orchestration.

Unlike traditional CRMs that passively store data, Otto **actively manages execution** by:

- Detecting which sales stage a lead is in
- Extracting actionable next steps from conversations
- Assigning tasks with SLAs and accountability
- Nudging team members when actions are overdue
- Providing executives with real-time intelligence

Core Innovation: Treating sales stages as executable modules (not labels), with configurable ownership that adapts to any org structure (solo operator → distributed team).

---

1. THE PROBLEM OTTO SOLVES

1.1 Manual Intervention Creates Execution Gaps

In home services (roofing, HVAC, plumbing, electrical), sales depends on:

- **CSRs (Customer Service Reps)** taking inbound calls, booking appointments
- **Sales Reps** going to homes for estimates, closing deals
- **Manual follow-up** that gets forgotten or lost

Common scenario:

- Customer calls: "Can you give me a quote for my leaky roof?"
- CSR books appointment for sales rep site visit
- Customer says: "Actually, let me talk to my wife first. Call me back at 4pm."
- CSR forgets or doesn't log it properly

- Lead is lost

1.2 Existing Tools Operate in Silos

Current landscape:

Tool Category	Examples	What They Do	What They DON'T Do
CRMs	Salesforce, ServiceTitan, Jobber, HubSpot	Store customer data, track pipeline stages	Extract intelligence from calls, enforce accountability, guide execution
Call Intelligence	Gong, Chorus, CallRail	Transcribe calls, detect sentiment/keywords	Convert analysis into assigned actions with SLAs
Field Sales Tools	Rilla, Siro	Record/analyze in-person meetings	Integrate with CSR activities, handle missed calls
AI Voice Agents	Bland AI, Synthflow	Answer calls with conversational AI	Integrate into stage-aware execution system

The gap: No tool connects CSR calls → field visits → follow-ups → executive oversight into one **execution-focused** platform.

1.3 No Platform Adapts to Org Structure

Existing tools assume fixed roles:

- ServiceTitan assumes separate "CSR" and "Field Tech" users
- Jobber assumes "Office Staff" and "Field Crew"

Reality in home services:

- **Solo operators** handle calls, estimates, AND installations themselves
- **Small teams** have CSRs who also do field work
- **Hybrid models** have dedicated appointment setters + field-only reps
- **Lead aggregators** qualify leads centrally, hand off to contractors

No existing system configures itself to these varied structures.

---

2. OTTO'S SOLUTION: END-TO-END FLOW

2.1 The 8 Canonical Sales Stages

Otto models the sales process as **8 discrete, executable stages:**

1. **Targeted Prospecting** - Initial lead identification/outreach
2. **Initial Contact & Rapport** - First meaningful interaction
3. **Discovery & Diagnosis** - Understanding customer problem
4. **Qualification** - Does customer/problem fit company criteria?
5. **Value Alignment** - Presenting solution and value
6. **Objections & Concerns** - Addressing blockers
7. **Decision & Commitment** - Closing the sale
8. **Delivery, Follow-Through & Expansion** - Fulfillment, upsell

Key innovation: These aren't just status labels - each stage has:

- Embedded logic for what actions should happen
- Configurable ownership (who is responsible)
- SLA defaults (how fast follow-ups should occur)
- Completion criteria (what must happen to progress)

2.2 Scenario: Customer Calls Arizona Roofers

Let's walk through how Otto works using a real example:

Customer: Sarah Johnson
Company: Arizona Roofers
Problem: Leaky roof

PATH A: Call Gets Answered by CSR

Step 1: Call Comes In

- Sarah calls Arizona Roofers' main number
- VoIP provider (CallRail/Twilio) sends webhook to Otto
- Otto waits 20 seconds for CSR to pick up

Step 2: CSR Answers

- CSR picks up, speaks with Sarah
- Call is recorded automatically
- VoIP provider sends recording URL to Otto after call ends

Step 3: AI Analysis (Shunya) Otto's AI engine analyzes the call recording:

- **Speech-to-Text:** Converts recording to transcript
- **Intent Detection:** "Customer wants roof inspection and quote"
- **Objection Detection:** None (customer is ready to book)
- **Sentiment Analysis:** Positive, cooperative
- **SOP Compliance:** Did CSR follow company script? Ask for address? Confirm availability?
- **Action Extraction:** "Book appointment for inspection at 123 Oak St, Thursday 2pm"

Step 4: Otto Stage Detection

```
INPUT: AI Analysis shows intent = "book_appointment"
       Lead = NEW (Sarah Johnson not in system)

LOGIC:
- This is a new lead (no existing ContactCard)
- Intent is clear: wants inspection/quote
- Set initial stage = "Discovery & Diagnosis" (skip stages 1-2 since customer came inbound)

CREATE:
- ContactCard (Sarah Johnson, phone, address)
- Lead (linked to ContactCard, stage = Discovery & Diagnosis)
- Appointment (scheduled for Thursday 2pm at 123 Oak St)
```

Step 5: CRM Integration (if enabled)

- Otto checks if ServiceTitan/Jobber integration is active
- Sends appointment details to CRM
- Syncs contact info bidirectionally

What CSR sees in UI:

- New lead created: Sarah Johnson
- Stage: Discovery & Diagnosis
- Next action: Sales rep needs to go to appointment Thursday 2pm

- Call recording available for review
- AI coaching: "Good job following SOP. Mentioned warranty early - customer responded positively."

PATH A (Alternative): CSR Call Results in Pending Lead

Modified Scenario: Sarah says "Let me check with my husband first. Call me back tomorrow at 4pm."

Step 3b: AI Analysis Detects Pending Action

- **Intent Detection:** "Interested but needs spousal approval"
- **Action Extraction:** "CALL BACK tomorrow at 4:00 PM"
- **Objection Detection:** "Decision-maker alignment" (common objection)

Step 4b: Otto Creates Pending Action

```
CREATE:
- ContactCard (Sarah Johnson)
- Lead (stage = Initial Contact & Rapport - not yet ready for Discovery)
- Action (
   type: "callback",
   description: "Call Sarah at 4pm tomorrow - needs to check with husband",
   owner: [assigned CSR or team],
   sla_deadline: Tomorrow 4:00 PM,
   priority: HIGH
  )
```

What CSR sees in UI:

- **Pending Actions Queue** (sorted by SLA deadline)
- Top of list at 3:00 PM tomorrow: "Call Sarah Johnson - DUE IN 1 HOUR"
- Lead status: Initial Contact (not booked yet)
- Call recording + transcript showing what Sarah said

Step 5b: SLA Nudge System

```
TIME: Tomorrow 3:00 PM (1 hour before deadline)
ACTION: Otto sends notification to CSR
MESSAGE: "Action due in 1 hour: Call Sarah Johnson at 4pm"
UI: Action jumps to top of queue, highlighted yellow

TIME: Tomorrow 4:00 PM (deadline reached)
ACTION: Otto escalates notification
MESSAGE: "URGENT: Action due NOW - Call Sarah Johnson"
UI: Action highlighted red

TIME: Tomorrow 4:30 PM (30 mins overdue)
ACTION: Otto notifies manager
MESSAGE: "Action overdue: CSR [name] hasn't completed callback to Sarah Johnson"
```

PATH B: Call Gets Missed (No CSR Available)

Step 1: Call Comes In

- Sarah calls, Otto waits 20 seconds for CSR pickup
- No CSR answers (after hours, overloaded, bathroom break)

Step 2: Voice Agent Picks Up (Immediately)

- Otto's AI voice agent answers BEFORE call drops
- Agent sounds like human receptionist: "Hi, thanks for calling Arizona Roofers! How can I help you?"
- Customer doesn't know they're talking to AI

Step 3: Voice Agent Qualifies Lead

```
AGENT SCRIPT (SOP-driven):
1. Personalization: "What's your name?" → "Hi Sarah, nice to meet you"
2. Problem Discovery: "What brings you to us today?" → "I have a leaky roof"
3. Qualification: "Is this your home or a rental?" → "My home"
4. Preferred Time: "When works best for someone to come take a look?" → "Thursday afternoon"

AI DECISION TREE:
- IF problem qualifies (roof repair = YES) → Continue
- IF problem doesn't qualify (outside service area = NO) → Politely decline
- IF customer hesitant → Schedule callback
```

Step 4: Voice Agent Creates "Ready for Booking" Lead

```
CREATE:
- ContactCard (Sarah Johnson, phone, preferred time: Thursday PM)
- Lead (stage = Discovery & Diagnosis, source = "Voice Agent")
- Flag: READY_FOR_BOOKING (agent completed qualification, needs human to confirm appointment)

OUTCOME: Voice agent says "Perfect! Someone from our team will call you back within the hour to confirm the exact time. Thanks Sarah!"
```

Step 5: CSR Sees "Ready for Booking" Queue

CSR Dashboard View:

```
┌─────────────────────────────────────────────┐
│ READY FOR BOOKING (Qualified by Voice Agent) │
├─────────────────────────────────────────────┤
│ Sarah Johnson                        │
│ Problem: Leaky roof                   │
│ Qualification: ✓ Homeowner, in service area │
│ Preferred Time: Thursday afternoon        │
│ Voice Recording: [PLAY]                │
│ Next Action: Call to confirm appointment    │
│ SLA: Call within 1 hour               │
└─────────────────────────────────────────────┘
```

Step 6: CSR Calls Sarah Back

- CSR calls: "Hi Sarah, this is [name] from Arizona Roofers. We got your inquiry about the roof. I have Thursday at 2pm open - does that work?"
- Sarah: "Perfect!"
- CSR books appointment in system
- Lead advances to stage "Discovery & Diagnosis" (now with confirmed appointment)

Fallback: If Voice Agent Can't Reach Customer

```
IF voice agent calls back and customer doesn't answer:
  → Send SMS: "Hi Sarah, this is Arizona Roofers. We tried reaching you about your roof inquiry. Reply with a good time to call back!"

IF customer still doesn't respond after 24 hours:
  → Create action for CSR: "Manual outreach needed - Sarah Johnson (missed call + no SMS response)"
```

PATH C: Sales Rep Goes to Field Appointment

Scenario: Sarah's appointment is confirmed for Thursday 2pm at 123 Oak St.

Step 1: Sales Rep Prepares

Sales Rep Mobile App View (Morning of Appointment):

```
┌──────────────────────────────────────┐
│ TODAY'S APPOINTMENTS                 │
├──────────────────────────────────────┤
│ 2:00 PM - Sarah Johnson              │
│ 123 Oak St, Phoenix AZ               │
│                                      │
│ [VIEW LEAD CONTEXT]                  │
│ - Initial call recording (from CSR)  │
│ - AI analysis: "Customer concerned about cost" │
│ - Preferred: Financing options       │
│ - Decision-maker: Needs husband approval │
│                                      │
│ [START NAVIGATION] [CALL CUSTOMER]   │
└──────────────────────────────────────┘
```

Key: Sales rep has FULL CONTEXT from prior CSR interaction. No information lost.

Step 2: Sales Rep Arrives On-Site

- Sales rep drives to 123 Oak St
- Otto mobile app detects geofence entry (50 meter radius around appointment address)
- **Recording starts automatically** (hands-free)

Geofencing Logic:

```
IF GPS location enters 50m radius of appointment.address:
  AND current_time within 1 hour of appointment.scheduled_time:
  THEN:
    - Start audio recording
    - Show notification: "Recording started - Sarah Johnson appointment"
    - IF Ghost Mode enabled: Recording only visible to sales rep, analysis sent to manager
```

Step 3: Sales Rep Conducts Meeting

- Sales rep inspects roof, discusses options with Sarah and her husband
- Conversation is recorded (if Ghost Mode OFF) or analysis-only (if Ghost Mode ON)

- Meeting lasts 45 minutes

Step 4: Sales Rep Leaves Geofence → Recording Stops

- Sales rep drives away, exits 50m radius
- Recording stops automatically
- File uploaded to Otto servers (Amazon S3)

Step 5: AI Analysis of Field Visit

Otto's AI analyzes the field meeting recording:

```
ANALYSIS OUTPUT:
- Outcome: PENDING (not closed, not rejected)
- Objections Detected:
  1. "Cost too high" (mentioned 3 times)
  2. "Want to get 2 more quotes" (comparison shopping)
- Pending Actions Extracted:
  1. "Bring shingle samples for Sarah to see" (she wants to visualize)
  2. "Send financing options by email" (husband wants to see 0% APR details)
- SOP Compliance: 85% (sales rep forgot to mention warranty - deviation from SOP)
- Performance Coaching:
  - What went well: "Built strong rapport, addressed timeline concerns effectively"
  - What could improve: "Mention warranty earlier (best performers bring it up in first 5 mins)"
```

Step 6: Sales Rep Sees Pending Actions

Mobile App View (After Meeting):

```
┌─────────────────────────────────────────┐
│ PENDING ACTIONS - Sarah Johnson         │
├─────────────────────────────────────────┤
│ ⚠️ Bring shingle samples (DUE: Friday)   │
│ ⚠️ Email financing options (DUE: Tomorrow) │
│                                         │
│ [MARK COMPLETE] [SNOOZE] [REASSIGN]     │
└─────────────────────────────────────────┘
```

Executive Dashboard View (Manager):

```
┌─────────────────────────────────────────────┐
│ PENDING LEADS - Action Required             │
├─────────────────────────────────────────────┤
│ Sarah Johnson - Stage: Objections & Concerns │
│ Owner: [Sales Rep Name]                     │
│ Objections: Cost, comparison shopping       │
│ Actions: 2 pending (DUE: Fri & Tomorrow)    │
│ Recording: [PLAY] (if not Ghost Mode)       │
│ Analysis: Available ✓                       │
└─────────────────────────────────────────────┘
```

Ghost Mode Protection:

```
IF sales_rep.ghost_mode_enabled == TRUE:
  THEN:
    - Recording stored but hidden from manager/exec
    - Only sales rep can play recording
    - Manager/exec see analysis only (objections, outcomes, SOP compliance)
    - Protects sales rep's proprietary techniques
```

2.3 How Stages Drive Everything

Stage as First-Class Primitive:

Each stage isn't just a label - it's a configuration with:

```
json
```

```json
{
  "stage_id": 3,
  "name": "Discovery & Diagnosis",
  "description": "Understanding customer problem and context",
  "default_sla": "24 hours",
  "completion_criteria": [
    "Problem fully documented",
    "Appointment scheduled OR quote provided"
  ],
  "typical_actions": [
    "Schedule site visit",
    "Send initial quote",
    "Gather additional info"
  ],
  "ai_focus": [
    "Detect problem type",
    "Identify urgency",
    "Check if qualified lead"
  ]
}
```

Example: Why Stage Context Matters

Same objection, different stages, different responses:

Stage	Customer Says	AI Analysis	Otto Action
Stage 2 (Initial Contact)	"Too expensive"	Premature objection - price not yet discussed	Action: "Build value before discussing price"
Stage 6 (Objections)	"Too expensive"	Legitimate pricing concern	Action: "Offer financing options, show ROI comparison"

Otto's AI knows which stage the lead is in and applies different logic accordingly.

2.4 Configurable Stage Ownership (Adapts to Any Org Structure)

Solo Operator (1 person owns all 8 stages):

```
Person: John (Owner/Operator)
Owns: Stages 1-8
Result: John sees ALL leads in ALL stages, handles everything
```

Small Team (CSRs handle calls, Sales Reps do field work):

```
Team A (CSRs): Emily, Mike, Sarah
Owns: Stages 1-3 (Prospecting, Initial Contact, Discovery)
Assignment: Round-robin by availability

Team B (Sales Reps): Tom, Lisa
Owns: Stages 4-8 (Qualification through Delivery)
Assignment: By territory (Tom = North Phoenix, Lisa = South Phoenix)
```

Hybrid Model (Dedicated qualifier + field reps):

```
Person: Jane (Lead Qualifier)
Owns: Stages 3-4 (Discovery & Qualification)
Result: Jane reviews ALL leads after initial contact, decides if qualified

Team C (Field Reps): Multiple people
Owns: Stages 5-8 (Value Alignment through Delivery)
Assignment: By skillset (premium vs. budget customers)
```

Configuration happens during onboarding:

```
ONBOARDING STEP 4: Stage Ownership Mapping

┌─────────────────────────────────────────────────────┐
│ Who owns which stages?                    │
├─────────────────────────────────────────────────────┤
│ Stages 1-2: ☑ Team: CSR Pool (3 people)       │
│           Assignment: Round-robin            │
│                                     │
│ Stage 3-4:  ☑ Person: Jane Doe              │
│           Assignment: Direct              │
│                                     │
│ Stages 5-8: ☑ Team: Sales Reps (2 people)      │
│           Assignment: By territory          │
└─────────────────────────────────────────────────────┘
```

---

3. CORE COMPONENTS & DATA FLOW

3.1 System Architecture

```
┌─────────────────────────────────────────────────────────┐
│      CUSTOMER INTERACTIONS                              │
│ (Inbound Calls, SMS, Outbound Calls, Field)             │
└────────────────────┬────────────────────────────────────┘
                     ↓
┌─────────────────────────────────────────────────────────┐
│    TELEPHONY/MESSAGING INTEGRATION                      │
│ - CallRail, Twilio, Dialpad, RingCentral                │
│ - Webhook verification & event processing               │
│ - Call recording retrieval                              │
└────────────────────┬────────────────────────────────────┘
                     ↓
┌─────────────────────────────────────────────────────────┐
│      OTTO AI INTELLIGENCE ENGINE                        │
│ (Proprietary - Built by Shunya)                         │
│                                                         │
│ Components:                                             │
│ - Speech-to-Text (ASR)                                  │
│ - Call Analysis (intent, sentiment, objections)         │
│ - SOP Compliance Scoring                                │
│ - Action Extraction Engine                              │
│ - Voice Agent (Missed Call Recovery)                    │
│ - RAG System (Ask Otto knowledge graph)                 │
│ - Reinforcement Learning Loop                           │
└────────────────────┬────────────────────────────────────┘
                     ↓ (Structured Intelligence)
┌─────────────────────────────────────────────────────────┐
│      OTTO STAGE ENGINE (Core Invention)                 │
│                                                         │
│ Stage State Machine:                                    │
│ - Current stage detection                               │
│ - Stage transition rules                                │
│ - Stage-specific context loading                        │
│                                                         │
│ Stage Ownership Manager:                                │
│ - Configurable stage→person/team mapping                │
│ - Dynamic assignment (workload, skill, territory)       │
│                                                         │
│ Action Generator:                                       │
│ - Stage-aware action creation                           │
│ - SLA initialization                                    │
│ - Priority scoring                                      │
```

```
│                                        │
│  Workflow Orchestrator:                │
│  - Task queue management               │
│  - Nudge/reminder system               │
│  - Escalation policies                 │
└────────────────────────────────────────┘
                    │
                    ↓
┌────────────────────────────────────────┐
│         USER INTERFACES                │
│  - CSR Web App (stage-based workspace) │
│  - Sales Rep Mobile App (field execution) │
│  - Manager Dashboard (oversight & intervention) │
│  - Ask Otto (natural language Q&A)     │
└────────────────────────────────────────┘
                    │
                    ↓
┌────────────────────────────────────────┐
│      DOWNSTREAM INTEGRATIONS           │
│  - CRM Sync (ServiceTitan, Jobber, etc.) │
│  - Job Management Systems              │
│  - Billing/Payment Systems             │
└────────────────────────────────────────┘
```

3.2 Core Data Models

ContactCard

- Primary entity representing a person (customer)
- Attributes: name, phone, email, address, source
- Linked to: Lead(s), Call(s), Appointment(s)

Lead

- Represents a sales opportunity
- Attributes: current_stage, stage_history, value_estimate, priority
- Linked to: ContactCard, Actions, Calls, Appointments

Call

- Represents a phone conversation
- Attributes: recording_url, transcript, duration, direction (inbound/outbound)
- Linked to: ContactCard, CallAnalysis

CallAnalysis

- AI-generated intelligence from call
- Attributes: intent, sentiment, objections[], sop_compliance_score, pending_actions[]
- Linked to: Call, Lead

Appointment

- Scheduled field visit
- Attributes: scheduled_time, address (for geofencing), status
- Linked to: Lead, RecordingSession

RecordingSession

- Field visit recording
- Attributes: geofence_entry_time, recording_url, ghost_mode_enabled
- Linked to: Appointment, RecordingAnalysis

RecordingAnalysis

- AI-generated intelligence from field meeting
- Attributes: outcome, objections[], pending_actions[], sop_compliance, performance_coaching
- Linked to: RecordingSession

Action

- Pending task that must be completed
- Attributes: description, owner, sla_deadline, status, stage
- Linked to: Lead

StageDefinition

- Template defining what each stage is
- Attributes: name, description, default_sla, completion_criteria, typical_actions
- Company-specific (configured during onboarding)

StageAssignment

- Maps people to stages they own
- Attributes: person_id, stage_id, assignment_type (direct/territory/round-robin)

MissedCallQueue

- Tracks missed calls pending voice agent recovery
- Attributes: call_id, status (PENDING/IN_PROGRESS/RESOLVED), sla_deadline
- Linked to: ContactCard

---

4. WHY VOIP & CRM INTEGRATIONS MATTER

4.1 VoIP Integration (Critical Infrastructure)

Why it's essential:

- Otto doesn't make/receive calls itself
- All calls route through existing VoIP provider (CallRail, Twilio, RingCentral, Dialpad)
- Otto needs webhooks + recording access to analyze calls

What Otto gets from VoIP:

```
WEBHOOK EVENT: call.started
{
  "call_id": "abc123",
  "from": "+1-555-0123",
  "to": "+1-555-COMPANY",
  "timestamp": "2024-06-15T14:30:00Z",
  "status": "ringing"
}

WEBHOOK EVENT: call.ended
{
  "call_id": "abc123",
  "duration": "320 seconds",
  "recording_url": "https://callrail.com/recordings/abc123.mp3",
  "disposition": "completed"
}
```

Multi-VoIP Support:

- Different companies use different providers
- Otto has adapter layer: `TelephonyProviderAdapter`
- Same internal API regardless of provider:

```
otto.telephony.get_recording(call_id)
→ Returns recording URL regardless if CallRail or Twilio
```

4.2 CRM Integration (Bidirectional Sync)

Why it matters:

- Home services companies already use ServiceTitan, Jobber, Housecall Pro
- They have years of customer data there
- Otto can't replace their CRM - must complement it

Data Flow INTO Otto (from CRM):

```
WHEN: Call comes in from phone number +1-555-0123

OTTO CHECKS:
1. Is this number in our ContactCard database?
   → NO

2. Query CRM: Is this an existing customer?
   → YES: ServiceTitan has customer "Sarah Johnson"
     - Address: 123 Oak St
     - Last service: 2023-03-15 (roof repair)
     - Payment status: Paid in full
     - Notes: "Prefers afternoon appointments"

OTTO CREATES:
- ContactCard with CRM data
- Sets lead context: "EXISTING CUSTOMER - upsell opportunity"
- Flags for CSR: "Last job was roof repair 1 year ago - possible maintenance check"
```

Data Flow OUT OF Otto (to CRM):

```
WHEN: Otto creates new lead + books appointment

OTTO SENDS TO CRM:
- New customer record (if not exists)
- Job details (roof inspection, 123 Oak St, Thursday 2pm)
- Source tracking (came from inbound call, voice agent qualified)
- Notes from call (customer wants financing options)

RESULT: CRM stays in sync, field techs see job in their CRM mobile app
```

Bidirectional Benefits:

- Otto doesn't duplicate customer data - leverages what's in CRM
- CRM stays current - appointments Otto books appear in CRM
- Reporting spans both systems - executives can see full picture

Supported CRMs:

- ServiceTitan
- Jobber
- Housecall Pro
- JobNimbus
- AccuLynx

---

5. LEAD NURTURING & FOLLOW-UP SYSTEM

5.1 How Actions Are Generated

AI Extraction from Conversations:

When Otto's AI analyzes a call/meeting, it extracts:

Explicit actions:

- Customer: "Call me back at 4pm"
- → Otto creates: Action(description="Call Sarah at 4pm", sla_deadline="4:00 PM")

Implicit actions:

- Customer: "I need to see some shingle samples first"

- → Otto creates: Action(description="Bring shingle samples to Sarah", sla_deadline="Before next meeting")

Follow-up sequences:

- Customer: "I'm getting 2 more quotes, I'll decide next week"
- → Otto creates:
  - Action 1: "Send comparison guide (why Arizona Roofers)" (DUE: Tomorrow)
  - Action 2: "Follow-up call to check decision" (DUE: Next Monday)

5.2 SLA Tracking & Nudge System

SLA Calculation:

```
PRIORITY RULES:

1. Explicit time reference:
   "Call me at 4pm" → SLA = 4:00 PM exactly

2. Urgency keywords:
   "As soon as possible" → SLA = 2 hours
   "This week" → SLA = Friday 5pm

3. Stage defaults:
   Stage 2 (Initial Contact) default SLA = 4 hours
   Stage 5 (Value Alignment) default SLA = 24 hours

4. No explicit time:
   Use stage default
```

Nudge Escalation:

```
ACTION CREATED: Call Sarah at 4pm tomorrow
SLA DEADLINE: 4:00 PM

TIMELINE:
- 3:00 PM (1 hour before): Notification to owner
  "Action due in 1 hour - Call Sarah Johnson"

- 4:00 PM (at deadline): Escalated notification
  "URGENT: Action due NOW"
  UI: Action highlighted red, jumps to top of queue

- 4:30 PM (30 mins overdue): Manager notification
  "Overdue action: CSR [name] hasn't called Sarah Johnson"

- 5:00 PM (1 hour overdue): Executive dashboard alert
  "High-priority action missed - Sarah Johnson callback"
```

5.3 Lead Nurturing Workflow

Example: Lead Goes Cold

Scenario: Sarah gets 3 quotes, doesn't decide immediately

Day 1: Sales rep visits, provides quote

- Action created: "Follow up in 3 days"

Day 4: Otto nudges sales rep

- Rep calls Sarah: "Hi Sarah, just checking if you had any questions about the quote"
- Sarah: "Still thinking about it, comparing with other quotes"
- AI analyzes call, extracts: "Comparison shopping" objection

Day 5: Otto generates new action based on AI recommendation

- Action: "Send ROI comparison document showing why Arizona Roofers > competitors"
- Source: AI identified that similar leads who received comparison docs closed 40% more often

Day 7: Otto schedules next follow-up

- Action: "Final follow-up call - if no interest, mark as lost"
- This prevents endless nurturing of dead leads

Otto's Intelligence:

- Tracks how long leads stay in each stage
- Identifies patterns: "Leads in Objections stage for >10 days rarely close"
- Suggests: "Focus on fresher leads, deprioritize Sarah"

---

6. ANALYSIS, STORAGE & INTELLIGENCE

6.1 What Gets Analyzed

Every customer interaction generates structured intelligence:

Call Analysis:

- Transcript (full text)
- Intent (what customer wants)
- Sentiment (positive/neutral/negative)
- Objections (price, timeline, quality, competition)
- SOP Compliance (did rep follow script?)
- Pending Actions (what needs to happen next)
- Decision-maker status (who has authority to buy?)

Field Meeting Analysis:

- Same as calls PLUS:
- Quote provided? (Yes/No/Pending)
- Customer reactions to price
- Technical issues discovered
- Next steps customer requested

6.2 How Analysis Is Stored

Structured Storage in Database:

```sql
```

```
call_analysis {
  id: uuid
  call_id: foreign_key
  lead_id: foreign_key

  -- Core Analysis
  intent: text ("book_appointment", "quote_request", "complaint")
  sentiment: enum (positive, neutral, negative)
  sentiment_score: float (-1.0 to 1.0)

  -- Objections (JSON array)
  objections: [
    {type: "price", severity: "medium", timestamp: "02:35"},
    {type: "timeline", severity: "low", timestamp: "04:12"}
  ]

  -- SOP Compliance
  sop_compliance_score: float (0-100)
  sop_deviations: [
    {step: "Ask for budget", compliant: false, note: "Skipped this question"}
  ]

  -- Actions Extracted
  pending_actions: [
    {description: "Call back at 4pm", sla_type: "explicit_time", confidence: 0.95},
    {description: "Send financing options", sla_type: "implicit", confidence: 0.80}
  ]

  -- Performance Coaching
  what_went_well: text
  what_could_improve: text

  created_at: timestamp
}
```

6.3 How Analysis Helps (Use Cases)

1. CSR/Sales Rep Level:

- See what worked/didn't work in their calls
- Get coaching without manager having to listen to every call
- Know exactly what to do next (pending actions auto-populated)

2. Manager Level:

- Identify top performers vs. underperformers
- See common objections across all calls
- Spot SOP compliance issues
- Intervene on high-value leads at risk

3. Executive Level (Ask Otto):

Query: "Why are my deals not closing?"

Otto's Answer (powered by knowledge graph):

```
Based on analysis of 47 lost deals in the last 30 days:

- 62% cited "price too high" as primary objection
- Only 15% of reps mentioned financing options proactively
- Top-performing rep (Tom) mentions "0% APR for 12 months"
  within first 5 minutes - his close rate is 40% higher

RECOMMENDATION:
Update SOP to require financing mention in Value Alignment stage
Train all reps on Tom's approach
```

Query: "Who are my best CSRs at converting callbacks to appointments?"

Otto's Answer:

```
Top 3 CSRs by callback→appointment conversion:

1. Emily: 78% conversion rate
   - Avg callback time: 1.2 hours (fastest response)
   - Uses empathy language: "I totally understand" appears 3x per call

2. Mike: 71% conversion rate
   - Offers 2 time options instead of asking "when works?"
   - Higher close rate on afternoon callbacks

3. Sarah: 65% conversion rate
   - Mentions warranty early (builds trust)

Company avg: 52%
```

6.4 Knowledge Graph & Reinforcement Learning

What's being built (by Shunya):

Traditional AI approach:

- Store call transcripts as text embeddings
- Search for "similar calls" when analyzing new call
- Generic recommendations

Otto's approach:

- **Knowledge graph** models relationships:

```
[SalesRep: Tom] -[HANDLED]→ [Objection: Financing] -[RESULTED_IN]→ [Outcome: Booked]
[SalesRep: Tom] -[USED_PATTERN]→ [Mentioned 0% APR early] -[LED_TO]→ [Outcome: Booked]
```

- **Enables complex queries:**
  - "Show me all reps who successfully handled financing objections AND closed within 3 days"
  - "What patterns do top performers use when customer says 'too expensive'?"
- **Reinforcement learning:**
  - When AI suggests an action and outcome is successful → Strengthen that pattern
  - When AI suggestion fails → Investigate why, adjust model
  - Continuous improvement based on real outcomes

Training Data:

- 7,000 real call transcripts
- 700,000 synthetic variations (10x expansion)
  - Vary objection types (price → timeline → quality)
  - Vary customer personas (budget-conscious → premium-seeking)
  - Vary outcomes (booked → callback → rejected)
- Hyper-contextualized to home services domain

---

7. SPECIAL FEATURES

7.1 Ghost Mode (Field Recording Privacy)

Problem: Sales reps don't want their techniques shared

- "My closing strategy is MY intellectual property"
- "If managers see my recordings, they'll steal my approach"

Solution: Ghost Mode Toggle

When sales rep enables Ghost Mode:

```
RECORDING STORED: YES (rep can review their own performance)
RECORDING VISIBLE TO MANAGER: NO (hidden, even from admin users)
ANALYSIS GENERATED: YES (outcomes, objections, SOP compliance)
ANALYSIS VISIBLE TO MANAGER: YES (exec gets insights without hearing call)
```

Technical Implementation:

```
IF recording.ghost_mode == TRUE:
  THEN:
    recording.accessible_by = [recording.sales_rep_id]  // Only rep
    recording.analysis.accessible_by = [ALL_MANAGERS]    // Everyone sees analysis

PERMISSION OVERRIDE:
  - Ghost Mode supersedes admin privileges
  - Even CEO can't access ghost mode recordings
```

Manager sees:

- Meeting outcome: "PENDING - needs to send financing options"
- Objections detected: "Price, comparison shopping"
- SOP compliance: 85%
- Coaching: "Consider mentioning warranty earlier"
- **Does NOT see:** Actual recording, exact words said

7.2 Ask Otto (Executive Intelligence)

Natural language queries across all company data:

Example Questions:

- "What are my pending leads?"
- "Why did we lose the Johnson deal?"
- "What are common objections this month?"
- "Who's my best closer for premium customers?"

- "Show me all leads stuck in Discovery stage for >7 days"

Data Sources Queried:

- All calls (transcripts + analysis)
- All leads (stage history, outcomes)
- Company documents (SOPs, pricing sheets)
- CRM data (customer history, payment info)

Role-Based Scoping:

```
MANAGER ROLE:
- Sees: All company data
- Can ask: "Why are we losing deals?"

CSR ROLE:
- Sees: All company calls + all leads
- Can ask: "What are my pending callbacks?"

SALES REP ROLE:
- Sees: Only their own data
- Can ask: "What's my close rate this month?"
```

Powered by same RAG system that analyzes calls

7.3 SOP Revision Proposals (Self-Improving)

How it works:

Step 1: Otto Detects Pattern

```
OBSERVATION:
- Sales rep "Tom" deviates from SOP
- SOP says: "Ask for budget in Discovery stage"
- Tom asks budget question in Value Alignment stage (later)
- Tom's close rate: 68%
- SOP-compliant reps' avg close rate: 42%
```

Step 2: Otto Flags Discrepancy

```
SYSTEM ALERT (to manager):
"SOP Deviation with Positive Outcome Detected"

Rep: Tom
Deviation: Delays budget question to Value Alignment
Result: 26% higher close rate than SOP-compliant approach

RECOMMENDATION: Consider revising SOP to match Tom's approach
```

Step 3: Manager Reviews

- Manager investigates: Listens to Tom's calls, compares with others
- Confirms Tom's approach is better
- Approves SOP revision

Step 4: Otto Generates Revised SOP

```
ORIGINAL SOP (Stage 3 - Discovery):
"Ask: What's your budget for this project?"

REVISED SOP (Stage 5 - Value Alignment):
"After presenting solution and ROI, ask: Does this align with what you were thinking budget-wise?"

Otto auto-generates updated SOP document (downloadable)
```

Why this matters:

- SOPs often written based on assumptions, not data
- Real-world performance reveals better approaches
- System learns and improves itself

---

8. AUTONOMOUS ONBOARDING

Goal: New company can go from signup → live in <1 hour

Step 1: Company Profile

- Company name, industry, service area
- Team size, structure (solo / small team / large)

Step 2: Telephony Connection

- Select VoIP provider (CallRail, Twilio, etc.)
- Otto provides webhook URL
- Company configures webhooks in provider dashboard
- Otto tests connection: "Make a test call to verify"

Step 3: SOP Upload

- Upload company documents (SOPs, scripts, service descriptions)
- Shunya analyzes and tags by stage
- Example: "Ask for referral" → Tagged as Stage 8 (Delivery/Expansion)

Step 4: Stage Configuration

- Review 8 default stages
- Rename if needed (keep actions the same)
- Set default SLAs per stage

Step 5: Team Member Assignment

- Add team members (name, email, role)
- Map to stages they own
- Set assignment logic (round-robin, territory, direct)

Step 6: Validation Checklist

```
✓ Telephony connected (test call successful)
✓ At least 1 team member assigned to each stage
✓ SOPs uploaded and analyzed
✓ Voice agent trained on company info
□ CRM integration (optional)

[GO LIVE] button enabled when all required items checked
```

Step 7: Go Live

- First inbound call routes through Otto
- Voice agent ready to answer missed calls
- Team sees new UI with stage-based workspace

9. WHAT MAKES OTTO DIFFERENT (COMPETITIVE MOAT)

9.1 vs. Traditional CRMs (ServiceTitan, Jobber, Salesforce)

Feature	Traditional CRM	Otto
Pipeline Stages	Passive labels	Executable modules with embedded logic
Data Entry	Manual	Auto-extracted from calls/meetings
Accountability	None	SLA tracking, nudges, escalation
Org Structure	Fixed roles (hardcoded)	Configurable stage ownership
Intelligence	Reports on past data	Real-time coaching + predictive actions

9.2 vs. Call Intelligence (Gong, Chorus)

Feature	Call Intelligence	Otto
Analysis	Transcripts, keywords, sentiment	Same PLUS stage-aware context
Actionability	Passive insights	Auto-generated assigned tasks with SLAs
Execution	None (just analytics)	Full workflow orchestration
Field Work	Calls only	Calls + in-person meetings

9.3 vs. AI Voice Agents (Bland AI, Synthflow)

Feature	Generic Voice Agent	Otto Voice Agent
Integration	Standalone	Embedded in stage-aware system
Training	Generic templates	Company-specific SOPs
Human Handoff	Manual or no handoff	Seamless → creates "ready for booking" queue
Follow-up	None	Pending actions auto-created for CSRs

9.4 Core Innovation Summary

What competitors can't easily replicate:

1. **Stage-Modular Architecture**
   - Stages as first-class primitives with embedded logic
   - Competitors locked into passive pipeline views
2. **Configurable Ownership**
   - Adapts to any org structure without code changes
   - Competitors assume fixed roles
3. **Hybrid AI-Human Orchestration**
   - AI provides intelligence, Otto enforces accountability
   - Competitors are either fully automated OR fully manual
4. **Voice-First Recovery**
   - Missed calls don't drop - voice agent picks up immediately
   - Competitors send generic SMS or log as "missed"
5. **Knowledge Graph + RL**
   - Graph-based learning from patterns, not just text similarity
   - Continuous improvement from real outcomes
6. **Unified Platform**
   - CSR calls + field visits + executive intelligence in ONE system
   - Competitors operate in silos

---

10. SUMMARY: THE OTTO FLYWHEEL

```
┌─────────────────────────────────────────┐
│ CUSTOMER INTERACTION                    │
│ (Call, SMS, Field Visit)                │
└─────────────────┬───────────────────────┘
    ↓
┌─────────────────────────────────────────┐
│ AI ANALYSIS                             │
│ (Intent, Objections, Actions, Coaching) │
└─────────────────┬───────────────────────┘
    ↓
```

```
┌─────────────────────────────────────────┐
│ STAGE DETECTION                         │
│ (Where is lead in sales cycle?)         │
└────────────────┬────────────────────────┘
                 ↓
┌─────────────────────────────────────────┐
│ ACTION GENERATION                       │
│ (Stage-specific tasks with SLAs)        │
└────────────────┬────────────────────────┘
                 ↓
┌─────────────────────────────────────────┐
│ OWNERSHIP ASSIGNMENT                    │
│ (Who's responsible based on stage config) │
└────────────────┬────────────────────────┘
                 ↓
┌─────────────────────────────────────────┐
│ EXECUTION                               │
│ (Human completes action, Otto tracks)   │
└────────────────┬────────────────────────┘
                 ↓
┌─────────────────────────────────────────┐
│ OUTCOME LOGGING                         │
│ (Success/failure fed back to AI)        │
└────────────────┬────────────────────────┘
                 ↓
┌─────────────────────────────────────────┐
│ REINFORCEMENT LEARNING                  │
│ (AI improves based on what worked)      │
└────────────────┬────────────────────────┘
                 ↓
    [Loop back to top - next interaction is smarter]
```

The result:

- Fewer missed opportunities (voice agent recovery)
- Faster follow-up (SLA nudges)
- Better execution (stage-specific actions)
- Continuous improvement (RL loop)
- Executive visibility (Ask Otto intelligence)

Otto doesn't just track sales - it executes sales.

APPENDIX: TECHNICAL SPECIFICATIONS

Tech Stack

- **Backend:** Python, FastAPI
- **Database:** PostgreSQL (relational), Redis (caching)
- **Queue:** Celery (background jobs)
- **Storage:** S3-compatible (call recordings)
- **AI/ML:** Shunya (on-prem deployment planned)
- **Telephony:** Multi-provider adapters (CallRail, Twilio, Dialpad, RingCentral)
- **CRM Integrations:** ServiceTitan, Jobber, Housecall Pro, JobNimbus, AccuLynx

Deployment

- Currently cloud-hosted
- Shunya moving on-prem for quality control
- Separate instances per customer (data isolation)

Data Security

- Call recordings encrypted at rest (S3)
- Ghost Mode enforces recording access controls
- Role-based permissions (CSR/Rep/Manager/Executive)
- HIPAA/PCI considerations for future (healthcare/payments)